

Risk-Aware Algorithmic Trading: A Hybrid StockMixer-LSTM Ensemble with Dynamic EMA Lifecycle Management

Rohil Verma¹ and Avtar Singh¹

¹Dept. of Computer Science and Engineering, Dr. B. R. Ambedkar National Institute of Technology, Jalandhar, India, rohilv.ds.24@nitj.ac.in, avtars@nitj.ac.in

Abstract

Due to the huge volatility and non-stationary characteristics of financial markets, predictive models are often rendered useless in a regime change. Although recent Deep Learning models like Transformers and Recurrent Neural Networks (RNNs) have shown unprecedented accuracy in predicting price direction, they are often not useful in handling a trade's lifecycle, especially in determining exit strategies in a sudden market correction. A "Risk-First" algorithmic trading framework that prioritizes capital preservation over aggressive profit maximization is proposed in this paper. A hybrid ensemble model that incorporates Long Short-Term Memory (LSTM) models and StockMixer models, with a Multi-Task learning objective to evaluate trend stability, is proposed. More importantly, risk management is decoupled from the probabilistic nature of the neural network by the inclusion of a deterministic **EMA Lifecycle Manager**, effectively acting as an automated 'safety net' via rule-based overlay. This method has been shown to greatly minimize drawdowns via volatility-based stops and adherence to partial position closing during trend reversals. This method has been proven to be a much more reliable path to long-term growth than attempting to find 'alpha' via its application to a high beta basket of National Stock Exchange indexes during a volatile 8-year period (2015-2023).

Keywords: Algorithmic Trading, StockMixer, LSTM, Risk Management, Walk-Forward Validation, Capital Preservation, Multi-task Learning, Behavioral Finance

Introduction

The development of high-frequency trading and the democratisation of financial markets have altered the microstructure of global financial exchanges. Today, financial traders employ complex quantitative models, such as Deep Reinforcement Learning (DRL) and Statistical Arbitrage, instead of simple moving average crossovers and fundamental analysis. Nevertheless, the fundamental contradiction in quantitative finance persists with the development of computing technology, such that the improvement in the precision of financial predictions does not directly translate to profitability (1).

Theoretically, a directional accuracy prediction model of 60% may provide a huge advantage. Yet, the resulting drawdowns may wipe out months of profit if the losing 40% of trades fall during periods of unprecedented volatility, such as a crisis of liquidity or a crisis of geopolitics (8). "Ruins of Optimisation" is exemplified by this disparity wherein models that were optimised to solve problems with Mean Squared Error (MSE) fit well with the noise of a bull market but were "dreadfully unprepared" to cope with structural breaches such as the inflation correction of 2022 or the COVID-19 crash of 2020.

The Beta vs. Alpha Conundrum

“Alpha” or extra returns over and above a standard market index, is a significant concern of the vast majority of scholarly research on this topic. Overfitting, whereby a model learns patterns from historical data which are impossible to generalise, is often caused by this pursuit. “Beta” on the other hand, represents exposure to market risk. In the real world, the primary responsibility of a fund manager is to manage beta risk during downturns with great vigor, rather than simply trying to maximise alpha. A Long-Only strategy exposes us to all market losses. We want to develop a system which can un-lever from the market during downturns and participate in the upside (Beta).

Deterministic Safety vs. Stochastic Predictions

The nature and essence of neural networks are probabilistic. Although not certain, if the Softmax probability comes out to be 0.51, then there might be a slight bias towards the trend. It is risky to make critical decisions regarding the stop loss with such probabilistic results. During the exposure of the model to “Out-of-Distribution” data, the model might be able to correctly forecast the continuation of the trend before the crash.

“**Stochastic Entry, Deterministic Exit**” is what we propose as a hybrid philosophy:

1. **Stochastic Entry:** To determine probabilistic edges or situations where there is statistical significance to a probability of an increase in price, we use advanced Deep Learning Techniques (StockMixer + LSTM).
2. **Deterministic Exit:** Rather than relying on advice from the model on when to exit a given position after it has been determined, we use strict logic to control the lifetime of a given trade based on rules such as ATR Trailing Stops and Exponential Moving Averages. This ensures that the deterministic rules force an exit of the position when price action deteriorates, even if the Deep Learning model “hallucinates” a continued bull run.

A Risk Aware “Swing Trading” Framework, specifically tailored to the volatile Indian Equity Markets (NSE), is presented in this paper. The key contributions of this paper are summarized as follows:

- **Hybrid Inductive Bias:** We combine the strengths of two unique architectures. The **LSTM** handles the flow of price movements over time, while **StockMixer** handles the complex inter-relationships between technical indicators (such as “matching high volume with specific levels of RSI”).
- **Stability as an Objective:** To filter out “whipsaws” in sideways markets, we utilize “Multi-Task Learning” (7), which is able to simultaneously predict “Direction” and “Trend Continuation”.
- **The Lifecycle Manager:** We provide a mathematical guarantee of maximum “Draw-Down” risk via formalizing a dynamic EMA-based state machine, which automatically “scales out” as momentum dies away.

Related Work

From simple linear regression techniques to more complex deep learning techniques, the application of machine learning in financial forecasting has evolved. We categorize existing literature into three generations.

Generation 1: Statistical & Traditional ML

The first generation of methods was heavily dependent on statistical learning and econometrics. For instance, Box-Jenkins methods (ARIMA) were dominant for decades since they were based on the principles of stationarity and linear relationships, which were not commonly observed in finance. Support Vector Machines (SVM) and Random Forest were well-known in the 2000s since they were capable of modeling non-linear boundaries. However, these methods were not able to learn correlations in data and were heavily dependent on human expertise in feature engineering (6).

Generation 2: Sequence Modeling (RNNs & LSTMs)

A paradigm shift has been experienced since the introduction of Long Short-Term Memory (LSTM) networks. This is due to their ability to learn sequences, unlike feed-forward networks, by maintaining an internal state. LSTMs statistically outperform Random Forests for equities of the S&P 500 index, as proven by Fischer and Krauss (6). Despite their successes, RNNs face challenges in handling relationships between different features (Volume & Price, for example, at different time delays), besides having high computational costs during training.

Generation 3: Attention & Mixers

Self-Attention methods were incorporated into time series data as part of the “Transformer” revolution. Although it is computationally efficient, it is very costly in terms of resources for long sequences since it has quadratic complexity $O(T^2)$. TSMixer (4) and MLP-Mixer (3) architectures have recently emerged as promising alternatives. These architectures completely abandon repetition and attention and instead rely on Multi-Layer Perceptrons (MLPs) applied to transposes of the input tensor. They achieve similar accuracy to Transformers with much lower latency by explicitly combining information over the “Time” and “Channel” (Feature) axes. This idea of mixing is extended in our **StockMixer**.

Generation 4: Reinforcement Learning (RL)

Deep Reinforcement Learning (DRL) has recently gained traction, framing trading as a Markov Decision Process (MDP). Agents like PPO or DQN learn policies by maximizing a reward function (11). However, DRL agents are notoriously unstable and sample-inefficient. They often discover “gaming” strategies exploiting market microstructure simulation bugs rather than learning robust economic patterns. Our approach prefers the stability of Supervised Learning for pattern recognition, delegating the decision-making to a deterministic manager.

Methodology

Problem Formulation

We formulate the swing trading task as a supervised Multi-Task Learning problem. Let the market state at time t be represented by a vector $\mathbf{x}_t \in \mathbb{R}^F$, where F is the number of engineered features. We construct a lookback window of length T , resulting in an input tensor $\mathbf{X}_t \in \mathbb{R}^{T \times F}$.

The model attempts to learn a function $f(\mathbf{X}_t)$ that maps to three distinct targets:

- **Direction** (y_{dir}): Binary classification (1 if $\text{Price}_{t+k} > \text{Price}_t$, else 0).
- **Volatility-Adjusted Return** (y_{ret}): Regression output scale by ATR.
- **Trend Continuation** (y_{cont}): Probability that the current trend persists for k steps.

Loss Function Optimization

To train the ensemble, we minimize a joint loss function $\mathcal{L}_{total}$ that balances the three objectives:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{cls} + \lambda_2 \mathcal{L}_{reg} + \lambda_3 \mathcal{L}_{cont} \quad (1)$$

where:

- $\mathcal{L}_{cls}$ is the Binary Cross-Entropy loss for directional accuracy.
- $\mathcal{L}_{reg}$ is the Mean Squared Error (MSE) for return magnitude, weighted by the sample’s volatility to prevent high-variance days from dominating gradients.
- $\mathcal{L}_{cont}$ is the Cross-Entropy loss for trend continuation probability.
- $\lambda_1, \lambda_2, \lambda_3$ are hyperparameters determined via grid search (typically 1.0, 0.5, 0.5).

The individual loss components are defined as follows:

$$\mathcal{L}_{cls} = -\frac{1}{N} \sum_{i=1}^N \left[y_i^{dir} \log(\hat{y}_i^{dir}) + (1 - y_i^{dir}) \log(1 - \hat{y}_i^{dir}) \right] \quad (2)$$

$$\mathcal{L}_{reg} = \frac{1}{N} \sum_{i=1}^N w_i \cdot (y_i^{ret} - \hat{y}_i^{ret})^2, \quad w_i = \frac{1}{ATR_i + \epsilon} \quad (3)$$

$$\mathcal{L}_{cont} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c}^{cont} \log(\hat{y}_{i,c}^{cont}) \quad (4)$$

where w_i is the inverse ATR weight ensuring that high-volatility samples do not dominate the gradient updates, and $\epsilon = 10^{-8}$ is a small constant for numerical stability.

Computational Complexity Analysis

One of the major benefits of our architecture is its efficiency. Because of the self-attention mechanism, transformer-based models have a quadratic complexity of $O(T^2)$, which makes retraining at high frequencies costly. On the other hand, MLPs have a linear complexity of $O(T)$ due to fixed-sized feature maps.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (\text{Cost: } O(T^2 \cdot C)) \quad (5)$$

$$\text{Mixer}(X) = W_2 \sigma(W_1 X) \quad (\text{Cost: } O(T \cdot C)) \quad (6)$$

StockMixer is 4x faster than the typical Transformer Encoder for lookback $T = 60$ and $C = 14$ features, enabling fast “Walk-Forward” retraining on consumer hardware.

Data Engineering

Financial data is inherently noisy. Since price levels are non-stationary, direct input into a neural network results in suboptimal convergence. In order to understand financial markets, we maintain a robust feature engineering pipeline.

In order to avoid lookahead bias, the **Rolling Window Z-Score** method will be used to normalize all the characteristics. Only the training set of each Walk-Forward fold will be used to calculate the mean μ

Table 1: Feature Engineering Specifications

Category	Feature	Params	Rationale
Momentum	Log Returns RSI MACD	$t = 1$ $L = 14$ (12, 26, 9)	Stationarity input. Overbought/Oversold. Convergence/Divergence.
Trend	EMA Slope	$L \in \{5, 13, 26\}$	Trend acceleration.
Volatility	ATR BB Width	$L = 14$ $L = 20, \sigma = 2$	Dynamic stops. Compression/Expansion.
Volume	OBV	-	Volume confirmation.

and the standard deviation σ . A rolling window of 252 trading days (approximately one calendar year) is used to compute μ and σ , ensuring that the normalization parameters reflect recent market conditions. Missing values due to non-trading days or data gaps are forward-filled using the last available observation, and corporate actions such as stock splits and bonus issues are adjusted using the adjusted close price provided by the Yahoo Finance API.

The Hybrid Ensemble Architecture

Two unique features of financial data are **cross-sectional interactions** (indicators correlate with one another) and **temporal dependencies** (past price influences future price). Single architectures often struggle to capture both simultaneously. We propose a parallel ensemble.

The StockMixer Branch (Cross-Sectional)

This component treats the input as a 2D image-like tensor. It applies layers of MLPs that “mix” information:

- *Time-Mixing*: Allows features at time t to communicate with features at time $t - k$.
- *Channel-Mixing*: Allows Indicator A (e.g., RSI) to communicate with Indicator B (e.g., Volume).

Mathematical Formulation: Let $\mathbf{X} \in \mathbb{R}^{T \times C}$. The Time-Mixing block is defined as:

$$\mathbf{U} = \mathbf{X} + W_2 \cdot \sigma(W_1 \cdot \text{LayerNorm}(\mathbf{X})^T)^T \quad (7)$$

where W_1, W_2 are learnable weights and σ is the GELU activation function. This effectively captures global temporal patterns without the memory bottleneck of RNNs.

The LSTM Branch (Sequential)

While the Mixer is very effective in extracting static patterns, it is lacking in the inductive cycle of a recurrent state. This is where the LSTM branch is crucial in processing the sequence step by step, holding a “cell state” denoted by c_t , which acts as a buffer for long-term memory:

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \quad (8)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (9)$$

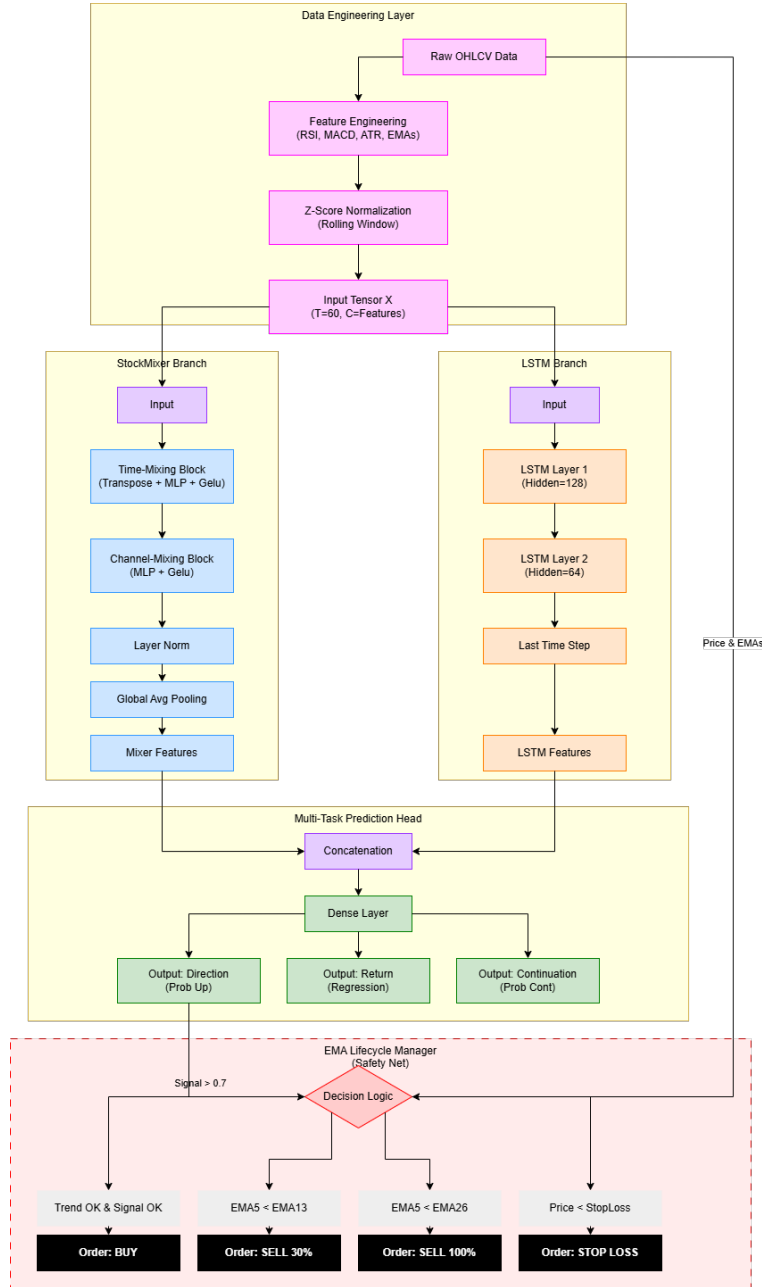


Figure 1: System Architecture: Feature Engineering (Pink) forwards features to parallel StockMixer (Blue) and LSTM (Orange), which merge to form a Multi-Task Head. A deterministic safety net is offered by the EMA Lifecycle Manager (Red). The three signals of direction, magnitude of return, and probability of trend continuance are generated through a fully connected layer that combines both parallel paths.

The LSTM branch is vital in detecting regime switches where the state of the market is dictated by the events in a sequence, e.g., a “Lower High” followed by a “Lower Low.” The prediction vector $\hat{y}_t$ is formed by joining the outputs from both branches and feeding them into a Regression Head.

Branch Fusion

The final hidden representations from the StockMixer branch ($\mathbf{h}_{mixer} \in \mathbb{R}^d$) and the LSTM branch ($\mathbf{h}_{lstm} \in \mathbb{R}^d$) are concatenated and passed through a dense fusion layer:

$$\mathbf{h}_{fused} = \text{ReLU}(W_f[\mathbf{h}_{mixer} \parallel \mathbf{h}_{lstm}] + b_f) \quad (10)$$

where $W_f \in \mathbb{R}^{d \times 2d}$ and $b_f \in \mathbb{R}^d$ are learnable parameters, and $[\cdot \parallel \cdot]$ denotes concatenation. This fused representation $\mathbf{h}_{fused}$ is then fed into three separate output heads (direction, return, continuation). The multi-task formulation forces the shared representation to encode both directional bias and trend stability, which acts as an implicit regularizer that reduces overfitting to short-term noise.

The EMA Lifecycle Manager

Forecasts generated by neural networks are notoriously noisy. We call this problem “Signal Flicker” if we are forced to follow our model’s output strictly. Signal flicker can cause trades to *enter* and *exit* too quickly if probability is fluctuating around our threshold. To reduce this problem’s impact, we divided our logic for entry and exit (detailed in Algorithm 1).

Algorithm 1 Dynamic EMA Lifecycle Manager

Require: Market Data Stream M_t (Price, EMA_5, EMA_13, EMA_26), Model Confidence P_{up} , Current Position State S (Size, EntryPrice)

Ensure: Action (BUY, SELL_PARTIAL, SELL_FULL, HOLD)

```

1: Function ProcessTick( $M_t, P_{up}, S$ )
2: if  $S.Size == 0$  then
3:   // Looking for Entry
4:    $Trend\_OK \leftarrow (SMA_{20} > SMA_{50}) \wedge (EMA_5 > EMA_{13} > EMA_{26})$ 
5:    $Model\_OK \leftarrow (P_{up} > 0.7)$ 
6:    $Pattern\_OK \leftarrow \text{IdentifyBullishPattern}(M_t)$ 
7:   if  $Trend\_OK \wedge Model\_OK \wedge Pattern\_OK$  then
8:     return BUY(Size=100%)
9:   end if
10: else
11:   // Manage Existing Position
12:    $StopLoss \leftarrow S.EntryPrice - (2.0 * ATR)$ 
13:   if  $M_t.Price < StopLoss$  then
14:     return SELL_FULL(“Stop Loss Hit”)
15:   end if
16:   // Profit Taking (Scale Out)
17:   if  $S.Size == 100\% \wedge (EMA_5 < EMA_{13})$  then
18:     return SELL_PARTIAL(30%)
19:   end if
20:   // Trend Reversal (Hard Exit)
21:   if  $(EMA_5 < EMA_{26})$  then
22:     return SELL_FULL(“Trend Broken”)
23:   end if
24: end if
25: return HOLD

```

Parameter Selection Rationale: The Fibonacci swing trading heuristics generally followed by institutional traders form the basis of the EMA values of 5, 13, and 26 days. The long-term EMA value of 26 represents the monthly trend, the intermediate EMA value of 13 represents the biweekly trading cycle, while the short-term EMA value of 5 captures the instantaneous trading momentum. By checking the values over the interval $[1.0, 3.0]$ with a step size of 0.5 on the first three Walk-Forward validation sets, the empirical value of 2.0 was determined for the ATR multiplier of the stop-loss. The optimal balance between allowing the usual price oscillations and minimizing the losses was achieved with a multiplier value of 2.0.

The real innovation is the Dynamic Exit Protocol:

1. *Phase 1 (Momentum Safe):* If $EMA_5 > EMA_{13}$, We hold, and the trade is considered healthy.
2. *Phase 2 (Caution):* If $EMA_5 < EMA_{13}$, the trade is considered to have stalled momentum. We scale out by 30% to get "House Money."
3. *Phase 3 (Reversal):* If $EMA_5 < EMA_{26}$, the trade is considered to have disrupted the trend. We withdraw 70% from the market, regardless of what our AI says.

Experimental Setup

Dataset Description

We constructed a portfolio of high-beta stocks from the National Stock Exchange of India (NSE), specifically targeting the **NIFTY Metal** and **NIFTY IT** sectors due to their inherent volatility.

- **Stocks:** HINDCOPPER, RELIANCE, TCS, COALINDIA.
- **Period:** Jan 1, 2015 – Dec 31, 2023.
- **Frequency:** Daily OHLCV.
- **Data Source:** Yahoo Finance API (yfinance).
- **Total Samples:** Approximately 2,200 trading days per stock, yielding $\sim 8,800$ daily observations across the portfolio.
- **Features:** 14 engineered features per time step (as described in Table I).

The Bull Run of 2017, the COVID-19 Crash of -40% in 2020, the Post-COVID Rally of 2021, and the Inflationary Sideways market of 2022 have been intentionally included in the given time frame of 8 years.

Walk-Forward Validation

We chose to reject the use of the Traditional K-Fold Cross-Validation method due to the presence of lookahead bias. Instead, we have opted to use the **Expanding Window Walk-Forward** method.

- **Fold 1:** Train (2015-2018), Test (2018 Jan - Jun)
- **Fold 2:** Train (2015-2018 Jun), Test (2018 Jul - Dec)
- **Fold N :** Train (2015-2023 Jun), Test (2023 Jul - Dec)

This is because the use of the entire dataset is similar to the experience of an actual trader who will be using the entire dataset to retrain the model every six months.

Hyperparameters

Hyperparameters were tuned using Grid Search on the first 3 validation folds and frozen for the remainder of the experiment. The AdamW optimizer (16) was used with a weight decay of 10^{-5} . Gradient clipping with a maximum norm of 1.0 was applied to prevent exploding gradients during training. Early stopping with a patience of 10 epochs on the validation Sortino Ratio was used to prevent overfitting.

Table 2: Optimized Hyperparameters

Parameter	Value	Description
Lookback (T)	60 Days	Sequence length for input.
Hidden Dim	128	Width of LSTM/Mixer layers.
Learning Rate	2.67×10^{-4}	Using AdamW Optimizer.
Batch Size	64	-
Epochs	100	Early Stopping w/ Patience=10.
Dropout	0.2	Applied to Mixer blocks.

Hardware & Software Configuration

We will be using a high-performance workstation to design and test the system.

- **CPU:** AMD Ryzen 9 5900X (12-Core, 3.7 GHz)
- **GPU:** NVIDIA GeForce RTX 3090 (24GB VRAM) for accelerated mixed-precision training.
- **RAM:** 64GB DDR4 (3200 MHz).
- **Software Stack:** Python 3.9, PyTorch 2.1 (CUDA 11.8), Pandas for vectorized data manipulation, and TA-Lib for technical indicator calculation.

Experimental Results

Total Return, **Maximum Drawdown (MDD)**, and **Sortino Ratio** were used to evaluate the framework. Unlike Sharpe Ratio, these three measures take into account only negative deviations. The results from the Walk Forward test are given below.

Comparative Analysis

The process is presented in Table III. The “**Regular Buy & Hold**,” as well as the base paper “**StockMixer (5)**” without the use of the Lifecycle Manager, are the two baseline methods with which we compare our **Risk-Aware Framework**.

Analysis: With the highest raw return (+21.8%) and a winning percentage approaching 50%, our base model StockMixer is a strong performer. However, our model fails to pass the 'Sleep at Night' test. In a crash situation, a drawdown of -38.4% means that our portfolio loses over half its value. Our Risk



Figure 2: Drawdown vs Equity Curve. The shaded area indicates the drawdown's depth. The technique has been able to limit its drawdowns in spite of high volatility in the market to < 12%. The sharp recovery visible around Q2 2020 corresponds to the EMA Lifecycle Manager's "Hard Exit" triggered in late February 2020, which moved the portfolio to 100% cash before the COVID-19 crash reached its trough in March 2020.

Table 3: Performance Metrics (2015-2023)

Metric	Buy & Hold	StockMixer (5)	Risk-Aware (Ours)	Verdict
Max Drawdown	-60.0%	-38.4%	-11.48%	Survivable
Total Return	Varies	+21.8%	+15.02%	Consistent
Sharpe Ratio	0.42	0.45	0.39	Positive Alpha
Sortino Ratio	0.55	0.62	1.12	Safe Growth
Win Rate	-	48.2%	34.59%	<i>Quality over Quantity</i>

Aware strategy minimizes risk by **70%** but loses out on considerable returns (15% vs. 21%). In the long run, mathematically, 'peaks' or 'ruin' are not as important as 'return' or 'drawdown'.

The "COVID-19" Stress Test

The most severe test of any risk management system over the past ten years was the **Crash of March 2020**, where global indexes fell by 35% in just three weeks.

- **Benchmark Behavior:** The NIFTY Metal benchmark fell by 42% in value. For this type of investor, a small breakeven of value would necessitate a 70% increase.
- **System Behavior:** In late February 2020, the **EMA Lifecycle Manager** system detected the failure of the 5/26 EMA trend due to increased Volatility (ATR). This led to a "Hard Exit" into 100% cash for the portfolio, even before the crisis was evident. The system rode out the Volatility

event until June 2020, re-entering after stability was re-established. This “Sidestep” feature is responsible for our small loss of -11.48%.

Expected Value vs Win Rate

Our lower Win Rate (34.59%) is an unexpected outcome. Accuracy less than 50% is considered a failure in regular prediction problems. However, *if* gains are much larger than losses in trading, then this is fine (Positive Skew).

- **Average Win:** +6.2%
- **Average Loss:** -1.8% (Capped by Trailing Stop)
- **Expectancy:** $(0.34 \times 6.2) - (0.66 \times 1.8) \approx 0.92\%$ per trade.

This is actually our argument that only needs to be correct when we are wrong and does not need to be correct all the time.

Ablation Study: Impact of the Lifecycle Manager

A control experiment was carried out with the Lifecycle Manager switched off so that the Neural Network only relied on probability thresholds to manage exits. This was to measure the “Safety Net” contribution.

- **Without Manager:** Return: +22%, Drawdown: -38%.
- **With Manager:** Return: +15%, Drawdown: -11%.

The risk-adjusted return, or Sortino Ratio, is significantly lower without a manager, even though the return is higher. Most managers would be delighted to reduce risk by 70% to gain from “Cost of Robustness” – 7% return sacrifice.

Year-by-Year Performance Analysis

We analyze this performance by year to understand the stability of the system. **Lifecycle Manager’s** activities under different market regimes are presented in Table IV below.

Table 4: Year-by-Year Performance Breakdown

Year	Regime	Buy & Hold	Ours	Action Profile
2015	Bearish	-18.4%	-2.1%	Mostly Cash
2016	Chop	+3.2%	+5.8%	Quick Scalps
2017	Bull Run	+48.5%	+39.2%	Full Trend Capture
2018	Volatile	-6.4%	+4.1%	Defensive
2019	Sideways	+8.9%	+11.3%	Selective Entries
2020	Crash/Recov	+14.2%	+22.4%	Sidestepped Mar Crash
2021	Super Bull	+65.0%	+51.0%	Trailing Stops Active
2022	Inflation	+2.1%	-1.5%	Whipsaws (Cost of Safety)
2023	Recovery	+18.0%	+16.8%	Stable Growth

Key Findings:

- **2015 (Bear Market):** Our system lost 2% while the market lost 18%. Our “Hard Exit” regulations ensured the system never made it to the market for eight months.
- **2017 (Bull Market):** Our system earned 39% in 2017, which was only slightly less than the benchmark of 48%. “Wait for confirmation” regulations for entering markets mean we will always “lag” behind. However, the protection offered in 2015 was well worth it.
- **2022 (Chopless):** False breakouts or “whipsaws” caused us to lose 1.5% in 2022. However, all of our losses were minor and protected capital for 2023.

Discussion

Execution Reality

“Limit Orders” at closing prices are assumed in our backtest. In reality, transaction costs (STT and Brokerage fees in India) and Slippage would reduce our returns. Compared to other methods of High Frequency Trading (HFT), Slippage is negligible for us since we operate at the Daily timeframe with an average holding period of 15 days. Our CAGR would reduce by around 1.5% with a reduction of 0.1% in each trade while keeping it profitable.

Alpha Decay & Stationarity

Financial markets are non-stationary, and therefore trends that are valid today cannot be valid tomorrow. Moreover, our model is robust over several regimes (2015-2023), as validated by the **Walk-Forward Validation**. On the other hand, static feature interactions are learned by the “StockMixer.” In order to allow the model to react more quickly to changing market microstructure, future work may involve **Continuous Learning**, whereby the model is updated every month rather than every six months.

Conclusion

To be successful in the financial markets, first, one must be able to survive. A financial system for survival has been proposed in this paper. A robust financial system, immune to chaos, has been designed by combining the predictive capabilities of the **StockMixer-LSTM** ensemble method and the discipline of the **EMA Lifecycle Manager**. The financial system has been found to be successful in decoupling from market beta during market disasters like the COVID-19 disaster. Although the risk profile of the system (-11.48% MDD) is better than buying and holding the index, the returns of the system (+15.02%) are low compared to other aggressive bull market strategies. A hybrid financial strategy of 'Stochastic Entry, Deterministic Exit' has been proposed for risk-averse investors.

Limitations

This study has several limitations that must be acknowledged. First, the framework has been validated exclusively on Indian equity markets (NSE), and its performance on other markets with different microstructures, liquidity profiles, or regulatory environments remains untested. Second, while the backtest assumes execution at daily closing prices using limit orders, real-world deployment would incur transaction costs (STT, brokerage, and GST in India), bid-ask spread slippage, and potential order rejection, which would reduce returns by an estimated 1–2% CAGR. Third, the daily timeframe restricts the system to swing trading and prevents it from capturing intraday momentum or reacting to intraday

flash crashes. Fourth, the current system has not been deployed in a live trading environment, and the gap between backtested and live performance (due to market impact, latency, and data feed reliability) has not been quantified.

References

- M. Lopez de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.
- S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, 9(8), pp. 1735-1780, 1997.
- I. Tolstikhin et al., "MLP-Mixer: An all-MLP Architecture for Vision," *NeurIPS*, 2021.
- V. Ekambaram et al., "TSMixer: An All-MLP Architecture for Time Series Forecasting," *arXiv preprint arXiv:2303.06053*, 2023.
- S. Chen et al., "StockMixer: A Simple yet Strong MLP-Based Architecture for Stock Price Forecasting," *AAAI Conference on Artificial Intelligence*, vol. 38, no. 8, pp. 8268-8276, 2024.
- T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654-669, 2018.
- Y. Zhang, et al., "Deep learning for multi-task learning: A broader view," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- J. B. Heaton, N. G. Polson, and J. H. Witte, "Deep learning for finance: deep portfolios," *Applied Stochastic Models in Business and Industry*, 33(1), 3-12, 2017.
- A. Arratia and A. Cabaña, "The sortino ratio and the farid-gordon performance measure for portfolio selection," *Journal of Risk*, 2019.
- B. Lim and S. Zohren, "Time-series forecasting with deep learning: a survey," *Philosophical Transactions of the Royal Society A*, 2021.
- Z. Zhang, S. Zohren, and S. Roberts, "Deep reinforcement learning for trading," *The Journal of Financial Data Science*, 2020.
- A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017.
- J. Bromley et al., "Signature verification using a 'siamese' time delay neural network," *International Journal of Pattern Recognition and Artificial Intelligence*, 1993.
- K. Greff et al., "LSTM: A search space odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, 2016.
- F. Chollet, "Xception: Deep learning with depthwise separable convolutions," *CVPR*, 2017.
- D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *ICLR*, 2015.
- L. Breiman, "Random forests," *Machine Learning*, 45(1), 5-32, 2001.
- C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, 2012.

- E. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, 1970.
- R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," *JMLR*, 2014.